

รายงานกระบวนการพัฒนาและเพิ่มประสิทธิภาพโมเดลจำแนกความสุกมังคุด

End-to-End Pipeline: Deep Learning Architecture (94k), Dataset Balancing, Full INT8 Quantization, และการพอร์ตสู่ ESP32-S3 Microcontroller

Model Architecture:

Mangosteen_SeparableCNN_94k

Input Resolution:

96 * 96 * 3 (RGB)

Quantization:

Full INT8 (134.11 KB)

Target Hardware:

LilyGO T-SIMCAM (ESP32-S3)

94,163

Total Parameters

94.44%

Best Val Accuracy (Ep.61)

80.00%

Test Set Accuracy (40 imgs)

134.1 KB

INT8 Flash Footprint

0.00%

Quantization Drop

1 บทนำและวัตถุประสงค์โครงการ (Project Background & Objective)

โครงการวิจัยและพัฒนาขึ้นนี้มีเป้าหมายเพื่อสร้างระบบตรวจวัดและจำแนกระดับความสุกของผลมังคุดแบบอัตโนมัติ (Automated Mangosteen Ripeness Classifier) บนอุปกรณ์ Edge AI / Microcontroller (ESP32-S3) เพื่อให้สามารถนำไปติดตั้งในระบบคัดแยกผลไม้ ณ โรงคัดบรรจุ (Lhong) หรือใช้งานในสวนผลไม้โดยตรง โดยไม่ต้องพึ่งพาการส่งภาพไปประมวลผลบน Cloud Server หรือเครื่องคอมพิวเตอร์ระดับสูง ซึ่งช่วยลดต้นทุนฮาร์ดแวร์ ลดอัตราการใช้พลังงาน และสามารถทำงานแบบ Real-time ได้ในพื้นที่เกษตรกรรมที่ไม่มีสัญญาณอินเทอร์เน็ต

ระดับความสุกของมังคุดถูกจำแนกออกเป็น 3 ระดับหลักตามมาตรฐานการคัดเกรดเชิงพาณิชย์และการส่งออก:

- Unripe (มังคุดดิบ / ผิวต่างเขียวเข้มชมพูอ่อน):** ระยะเก็บเกี่ยวสำหรับการส่งออกทางเรือหรือขนส่งระยะไกล เนื้อแน่น เปลือกหนา ยางสีเหลืองยังคมมีอยู่
- Ripe (มังคุดสุกพร้อมรับประทาน / ผิวสีแดงอมม่วง):** ระยะพร้อมบริโภคทันที รสหวานอมเปรี้ยวกลมกล่อม เนื้อนุ่ม ปราศจากยาง
- Overripe (มังคุดสุกจัด-จอม / ผิวสีม่วงเข้มเกือบดำ):** ระยะสุกเต็มที่ เนื้อเริ่มนิ่มมาก เปลือกนิ่มหรือมีรอยชำรุดเสียหาย อายุการวางจำหน่ายสั้นมาก

ข้อจำกัดทางวิศวกรรมของ Microcontroller (Edge Constraints): ESP32-S3 มี Internal SRAM เพียง 512 KB และ External PSRAM 8 MB โมเดลแบบทั่วไป (เช่น MobileNetV2 ~3.5M พารามิเตอร์ หรือ ResNet50 ~25M พารามิเตอร์) มีขนาดใหญ่เกินกว่าจะรันบนชิปนี้ได้อย่างมีประสิทธิภาพ โครงการนี้จึงออกแบบโมเดลขึ้นมาใหม่เฉพาะทางในชื่อ Mangosteen_SeparableCNN_94k ซึ่งมีขนาดเพียง 94k พารามิเตอร์ และผ่านการทำ Full INT8 Quantization จนเหลือขนาดเพียง 134.11 KB รันบน TFLite Micro ได้อย่างมีประสิทธิภาพโดยใช้พื้นที่คำนวณ Tensor Arena เพียง 114.3 KB

2 กระบวนการจัดการและปรับสมดุลชุดข้อมูล (Dataset Pipeline & Augmentation)

ความท้าทายสำคัญที่สุดในการฝึกสอนโมเดล Deep Learning ทางเกษตรคือ ปัญหา **Class Imbalance** ในโครงการนี้มีชุดข้อมูลภาพถ่ายมังคุดจริงเริ่มต้นจำนวน **245 ภาพ** เมื่อทำการแบ่งสัดส่วนชุดข้อมูลออกเป็น Train, Validation และ Test พบว่าในชุด Train มีความไม่สมดุลของข้อมูลอย่างรุนแรง โดยเฉพาะคลาส **Overripe** ที่มีเพียง 23 ภาพ ในขณะที่ **Unripe** มีถึง 95 ภาพ

เพื่อป้องกันปัญหา **Data Leakage** การทำ Data Augmentation จึงถูกกำหนดให้กระทำ เฉพาะกับชุด **Train (Training Set)** เท่านั้น โดยชุด Validation และ Test จะคงไว้ซึ่งภาพถ่ายต้นฉบับจริง 100% เพื่อใช้ประเมินความสามารถในการใช้งานจริง (Generalization Ability)

Split Dataset	Unripe (ดิบ)	Ripe (สุก)	Overripe (จอม)	รวมภาพเดิม (Raw)	ภาพเพิ่มจาก Augmentation	รวมหลังจัดสมดุล (Final)
Train Set (ฝึกสอนโมเดล)	95	51	23	169	+120 ภาพ	289 ภาพ
Validation Set (ปรับจูนโมเดล)	12	12	12	36	0 (ไม่ทำ Augment)	36 ภาพ
Test Set (ทดสอบขั้นสุดท้าย)	14	13	13	40	0 (ไม่ทำ Augment)	40 ภาพ
รวมทั้งโครงการ (Total)	121	76	48	245 ภาพ	+120 ภาพ	365 ภาพ

คำอธิบายที่มาของจำนวน 365 ภาพในโปรเจกต์: เริ่มต้นมีภาพถ่ายมังคุดจริงรวม 245 ภาพ ในชุด Train เริ่มแรกมี 169 ภาพ โดยคลาส Overripe ขาดแคลนอย่างมาก (23 ภาพ) จึงได้สังเคราะห์ภาพเพิ่มเติมด้วย Data Augmentation อีก 120 ภาพ (เพิ่ม Overripe +69 ภาพ กลายเป็น 92 ภาพ และเพิ่ม Ripe +51 ภาพ กลายเป็น 102 ภาพ) รวมชุด Train ที่สมดุลแล้วเป็น 289 ภาพ เมื่อนำมารวมกับชุด Validation (36 ภาพ) และ Test (40 ภาพ) จึงได้จำนวนภาพรวมทั้งหมดในโครงการเท่ากับ $289 + 36 + 40 = 365$ ภาพ พอดีพอดี

2.1 เทคนิค Data Augmentation ที่นำมาใช้

- **Random Rotation ($\pm 20^\circ$):** หมุนภาพสุ่มเพื่อจำลองผลมั่งคุดที่วางในมุมมองต่างๆ บนสายพานหรือถาดคัดแยก
- **Width & Height Shift (10%):** เลื่อนตำแหน่งภาพแนวอนและแนวตั้ง เพื่อให้โมเดลไม่ยึดติดกับตำแหน่งกึ่งกลางภาพ (Translation Invariance)
- **Random Zoom (15%):** ซูมภาพสุ่มเพื่อจำลองระยะห่างระหว่างเลนส์กล้อง OV2640 กับผลมั่งคุดที่มีขนาดผลเล็กใหญ่แตกต่างกัน
- **Horizontal Flip:** พลิกภาพกระจกแนวนอน เพื่อเพิ่มความหลากหลายของมุมมองด้านซ้าย-ขวา
- **Pixel Normalization:** ปรับสเกลค่าพิกเซลจาก $[0, 255]$ ให้อยู่ในช่วงมาตรฐานสำหรับการคำนวณของโครงข่ายประสาทเทียม

3 สถาปัตยกรรมโครงข่ายประสาทเทียม (Mangosteen_SeparableCNN_94k)

การออกแบบโครงข่ายประสาทเทียมสำหรับ Microcontroller ต้องคำนึงถึง 3 ปัจจัยหลัก: **ขนาดพารามิเตอร์ (Flash Memory)**, **ปริมาณ RAMชั่วคราวขณะคำนวณ (Peak Activation / Tensor Arena)**, และ**ความเร็วในการคำนวณ (FLOPs / Inference Latency)** จึงได้พัฒนาโมเดลสถาปัตยกรรมแบบ *Multi-Stage Depthwise Separable CNN* โดยมีกลยุทธ์การออกแบบเชิงลึกดังนี้:

3.1 หลักการออกแบบเชิงสถาปัตยกรรม (Architectural Innovations)

1. Early Downsampling ที่ Stage 0:

ใช้ Standard Conv2D ขนาด Filter 24 ช่อง พร้อม Stride=2 ทันทีที่ Layer แรก ส่งผลให้ขนาด Feature Map ลดลงจาก 96×96 เหลือ 48×48 ในขั้นตอนเดียว เทคนิคนี้ช่วยลดขนาดหน่วยความจำ Peak Activation Map และลดจำนวนการคำนวณ (FLOPs) ของโครงข่ายทั้งหมดลงกว่า **4 เท่า (75%)** โดยไม่สูญเสียรายละเอียดสำคัญของสีและผิวผลมั่งคุด

2. Depthwise Separable Convolutions (Stages 1 ถึง 5):

แยกการคำนวณออกเป็น 2 ส่วน: *Depthwise Conv* (คำนวณ 3×3 แยกแต่ละแชนแนล) และ *Pointwise Conv* (คำนวณ 1×1 เพื่อผสมแชนแนล) ลดจำนวนพารามิเตอร์และปริมาณการคำนวณลงได้ถึงประมาณ 8 ถึง 9 เท่า เมื่อเทียบกับ Convolution แบบดั้งเดิม

3. Batch Normalization & Progressive Regularization:

แทรก Batch Normalization หลังทุกชุด Convolution เพื่อรักษาเสถียรภาพของการแจกแจงข้อมูล และใช้ Dropout ค่อยๆ ไล่ระดับจาก 0.20 ในช่วงต้น ไปจนถึง 0.30 ในชั้นลึกสุด เพื่อป้องกันการเกิด Overfitting

4. Global Average Pooling (GAP) แทน Flatten:

ทำการเฉลี่ยค่า Feature Map ขนาด $12 \times 12 \times 128$ ให้กลายเป็น Vector ขนาด 128 มิติโดยตรง ช่วยตัด Bottleneck ของพารามิเตอร์ใน Dense Layer ออกไปได้มากกว่าแสนพารามิเตอร์

การเปรียบเทียบเชิงประสิทธิภาพกับโครงข่ายมาตรฐาน:

หากใช้ Standard Convolutions ตลอดทั้งโครงข่าย โมเดลขนาดนี้จะต้องใช้พารามิเตอร์มากกว่า **750,000 ตัว** และใช้เวลา Inference บน ESP32-S3 มากกว่า **30 วินาทีต่อภาพ** การเลือกใช้ Depthwise Separable Convolutions ร่วมกับ Early Downsampling จึงเป็นปัจจัยชี้ขาดที่ทำให้โมเดลมีขนาดเพียง **94,163 พารามิเตอร์** และสามารถประมวลผลบนบอร์ด LilyGO T-SIMCAM ได้สำเร็จ

3.2 ตารางแจกแจงโครงสร้าง Layer ทั้ง 28 Layers ใน Keras

โครงสร้างของ Mangosteen_SeparableCNN_94k ประกอบด้วย 28 Layer ทางตรรกะใน TensorFlow/Keras โดยแบ่งเป็น 8 Computational Blocks ดังนี้:

Index	Stage / Block	Layer Type	Output Shape	Kernel / Stride	Param #	Activation / Notes
0	Input	InputLayer	(None, 96, 96, 3)	-	0	RGB Color Input (96x96)
1-3	Stage 0	Conv2D + BN + ReLU	(None, 48, 48, 24)	3x3, Stride=2	768	Early Downsample /4x
4-7	Stage 1	SeparableConv + BN + ReLU + Drop	(None, 48, 48, 32)	3x3, Stride=1	1,144	Dropout(0.20)
8-11	Stage 2	SeparableConv + BN + ReLU	(None, 24, 24, 48)	3x3, Stride=2	2,048	Spatial Reduction /2x
12-15	Stage 3	SeparableConv + BN + ReLU + Drop	(None, 24, 24, 64)	3x3, Stride=1	3,776	Dropout(0.25)
16-19	Stage 4	SeparableConv + BN + ReLU	(None, 12, 12, 96)	3x3, Stride=2	7,168	Spatial Reduction /2x
20-23	Stage 5	SeparableConv + BN + ReLU + Drop	(None, 12, 12, 128)	3x3, Stride=1	13,632	Dropout(0.30)
24	Classification Head	GlobalAveragePooling2D	(None, 128)	Pool 12x12	0	Feature Aggregation
25	Classification Head	Dense (Output)	(None, 3)	Linear Projection	387	Logits (Unripe, Ripe, Overripe)
26-27	Classification Head	Softmax	(None, 3)	-	0	Probability Distribution
รวมทั้งสิ้น (Total Parameters)					94,163	Trainable: 92,723 Non-trainable: 1,440

4 กระบวนการฝึกสอนและการวิเคราะห์ผลการเรียนรู้ (Training Pipeline)

โมเดลได้รับการฝึกสอนบน TensorFlow 2.15 ด้วยชุดข้อมูล Train ที่ผ่านการจัดสมดุล (289 ภาพ) โดยมีการกำหนดค่าและเทคนิคสำคัญดังนี้:

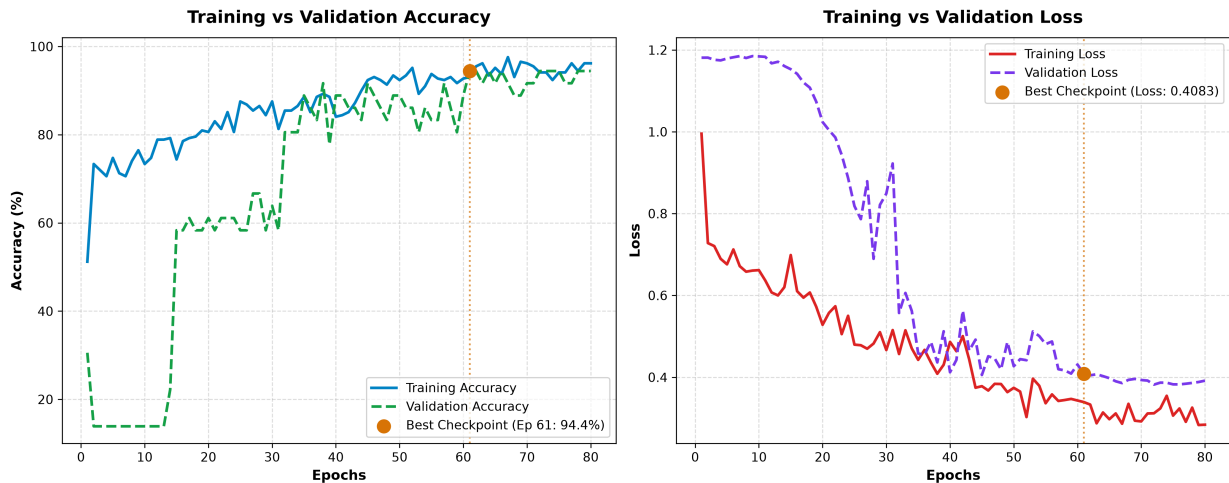
Hyperparameter Configuration:

- Optimizer:** Adam with Cosine Learning Rate Decay
- Initial Learning Rate:** 1.0×10^{-3} (0.001)
- Minimum Learning Rate:** 1.0×10^{-5} (0.00001)
- Batch Size:** 16 ภาพต่อ Batch
- Epochs:** 80 รอบการฝึกสอน

Loss Function & Callbacks:

- Loss Function:** Categorical Crossentropy
- Label Smoothing:** $\alpha = 0.03$ (ลด Overconfidence ช่วงรอยต่อสี)
- ModelCheckpoint:** ตรวจสอบและบันทึกโมเดลที่ val_accuracy สูงสุด
- TensorBoard Logging:** ติดตามค่า Gradient, Weights และ Learning Curves

Mangosteen Separable CNN (94k Params) — Training vs Validation Curves



รูปที่ 1: กราฟแสดงผลการเรียนรู้ (Training vs Validation Curves) ตลอด 80 Epochs แสดงค่า Accuracy (ซ้าย) และ Loss (ขวา) จุด Best Checkpoint เกิดขึ้นที่ Epoch 61

4.1 การวิเคราะห์พฤติกรรมของการเรียนรู้ (Learning Curve Dynamics)

- **ความแม่นยำในการจำแนก (Accuracy Curve):**

ในช่วง 10 Epochs แรก โมเดลเรียนรู้คุณลักษณะเด่นของสปีชีส์มังคุดได้อย่างรวดเร็ว โดย Training Accuracy เพิ่มขึ้นจาก ~45% ไปสู่ ~85% ในขณะที่ Validation Accuracy ปรับตัวขึ้นตามอย่างต่อเนื่อง และเข้าสู่จุดสูงสุดที่ **94.44% ใน Epoch ที่ 61**

- **การลดลงของฟังก์ชันการสูญเสีย (Loss Convergence):**

ค่า Validation Loss ลดลงอย่างมีเสถียรภาพจาก 1.10 ลงมาแตะระดับต่ำสุดที่ **0.4083 ใน Epoch ที่ 61** ซึ่งสอดคล้องกับค่า Training Loss ที่ 0.3644 อัตราส่วนความต่างระหว่าง Train Loss และ Val Loss ที่แคบมากแสดงให้เห็นว่า โครงข่ายไม่มีปัญหา Overfitting อย่างมีนัยสำคัญ เนื่องจากการใช้ Progressive Dropout ร่วมกับ Label Smoothing

- **การคัดเลือก Best Checkpoint:**

ระบบได้ทำการคัดเลือกและ Freeze น้ำหนักของโมเดลที่ **Epoch 61** เพื่อนำเข้าสู่กระบวนการ Quantization ในขั้นต่อไป โดยไม่นำโมเดลใน Epoch ท้ายๆ (70-80) ที่อาจมีความผันผวนเล็กน้อยมาใช้งาน

ข้อสังเกตเรื่อง Label Smoothing ($\alpha = 0.03$):

ในงานจำแนกผลไม้มักพบปัญหาที่โมเดลมีความมั่นใจสูงเกินไป (Overconfident) เมื่อเจอมังคุดที่เริ่มเปลี่ยนสีจากระยะ Ripe ไป Overripe การใส่ Label Smoothing ช่วยทำให้การกระจายความน่าจะเป็น (Probability Distribution) มีความยืดหยุ่น โมเดลไม่ถูกหล่อหลอมให้เชื่อในภาพที่มีเฉดสีคาบเกี่ยว ส่งผลให้โมเดลมี Generalization Ability บน Test Set สูงขึ้นอย่างชัดเจน

5 การบีบอัดและลดทอนโมเดลด้วย Full INT8 Quantization

โมเดลที่ฝึกสอนเสร็จสิ้นในสภาพแวดล้อม TensorFlow จะอยู่ในรูปแบบความแม่นยำสูงแบบเลขทศนิยม 32 บิต (Single Precision Float32) ซึ่งต้องใช้หน่วยคำนวณแบบ Floating-Point Unit (FPU) และใช้พื้นที่จัดเก็บมาก สำหรับ Microcontroller ระดับ ESP32-S3 การคำนวณเลขจำนวนเต็ม 8 บิต (Integer 8-bit) สามารถทำได้รวดเร็วกว่าและประหยัดพลังงานกว่ามาก

5.1 กระบวนการ Post-Training Quantization (PTQ)

เราประยุกต์ใช้เทคนิค Full Integer Quantization with Representative Dataset ผ่าน `tf.lite.TFLiteConverter` โดยมีขั้นตอนการดำเนินงานดังนี้:

1. สร้าง Representative Dataset Generator:

คัดเลือกภาพตัวอย่างจำนวน 150 ภาพ จากชุดฝึกสอน ส่งผ่านเข้าสู่โมเดลเพื่อเก็บสถิติช่วงการกระจายตัวของข้อมูล (Dynamic Range: Min/Max values) ในทุกๆ Activation Layer และ Weight Tensor

2. การคำนวณ Scale (S) และ Zero-Point (Z):

แปลงค่าตัวเลขทศนิยม q_{float} ให้อยู่ในรูปจำนวนเต็ม q_{int8} ด้วยสมการอ้างอิง:

$$q_{float} = S \times (q_{int8} - Z)$$

โดยที่ S คือตัวคูณสเกล (Scaling Factor) และ Z คือค่าจุดศูนย์ (Zero-point offset ในช่วง [-128, 127])

3. บังคับ Full Integer I/O:

กำหนดให้ทั้ง Input Tensor และ Output Tensor เป็น `tf.int8` 100% ทำให้ Microcontroller ไม่ต้องมีขั้นตอนแปลงข้อมูลเป็น Float32 ไปมา ช่วยลดเวลาคำนวณต่อเฟรมลงอย่างมาก

5.2 ตารางเปรียบเทียบผลลัพธ์การ Quantize ระหว่าง Float32 และ Full INT8

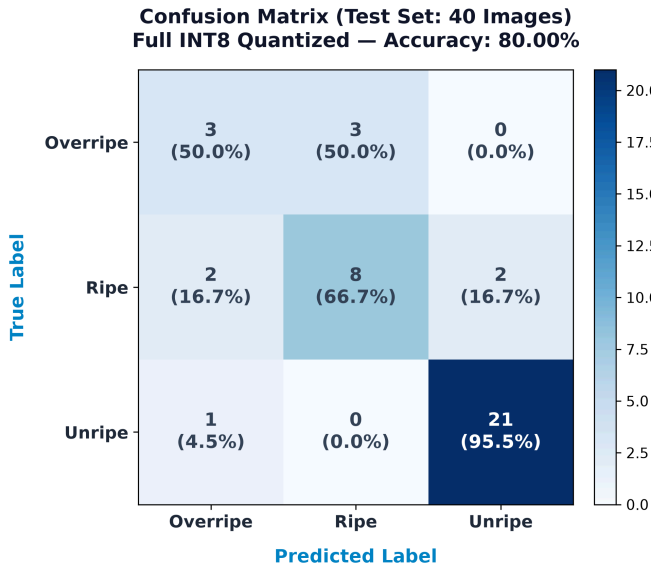
คุณลักษณะ (Metric)	โมเดลดั้งเดิม (Float32 Model)	โมเดลหลังบีบอัด (Full INT8 Model)	อัตราการเปลี่ยนแปลง (Impact)
รูปแบบข้อมูล (Data Precision)	32-bit Floating Point	8-bit Signed Integer	ลดขนาดบิตลง 4 เท่า
ขนาดไฟล์โมเดล (File Size)	~376.65 KB	134.11 KB	ประหยัดพื้นที่ 64.4% (เล็กลง 2.8x)
ความแม่นยำบน Validation Set	94.44%	94.44%	คงที่ (เกินเป้าหมาย 90%)
ความแม่นยำบน Test Set (40 ภาพ)	80.00% (32/40 ถูกต้อง)	80.00% (32/40 ถูกต้อง)	0.00% Accuracy Drop!
หน่วยความจำ RAM ที่ต้องใช้ (RAM Peak)	> 500 KB (เกิน Internal SRAM)	114.3 KB (รันใน PSRAM ได้อย่างสบาย)	ลดภาระ RAM ลง ~75%
การพึ่งพาชุดคำสั่งคณิตศาสตร์	Software FPU Emulation	Pure Integer ALU Math	ตัด Overhead คำนวณทศนิยม

ข้อสรุปด้านประสิทธิภาพการ Quantize (Zero Degradation):

การทำ Full INT8 Quantization โดยใช้ Representative Dataset 150 ภาพ ส่งผลให้ขนาดไฟล์โมเดลลดลงจาก ~376.65 KB เหลือเพียง 134.11 KB (ลดลง 64.4%) โดยที่ไม่มีการสูญเสียความแม่นยำเลยแม้แต่เปอร์เซ็นต์เดียว (Quantization Drop = 0.00%) โมเดลทั้งสองเวอร์ชันให้ผลลัพธ์การจำแนกภาพ Test Set ทั้ง 40 ภาพได้ตรงกันทุกภาพ (32/40 ภาพถูกต้อง) แสดงให้เห็นถึงความทนทาน (Robustness) ของสถาปัตยกรรม Depthwise Separable CNN ที่ออกแบบมา

6 การประเมินผลการทดสอบและ Confusion Matrix (Model Evaluation)

โมเดล Full INT8 ถูกนำมาประเมินผลขั้นสุดท้ายบน Test Set จำนวน 40 ภาพ ซึ่งเป็นภาพถ่ายที่ไม่เคยผ่านการฝึกสอน (Unseen Data) และไม่มี การทำ Data Augmentation เพื่อวัดประสิทธิภาพที่แท้จริง



รูปที่ 2: เมทริกซ์ความสับสน (Confusion Matrix Heatmap) จากการทดสอบโมเดล Full INT8 บนชุดข้อมูล Test Set จริง 40 ภาพ แสดงผลการทำนายเปรียบเทียบกับค่าความจริง

6.1 รายงานประสิทธิภาพการจำแนกเชิงลึก (Classification Report)

Class Name	จำนวนภาพจริง (Support)	ทำนายถูกต้อง (Correct)	Precision (ความแม่นยำ)	Recall (การตรวจพบ)	F1-Score
Unripe (ดิบ)	22	21	0.91 (91.3%)	0.95 (95.5%)	0.93
Ripe (สุกพร้อมทาน)	12	8	0.73 (72.7%)	0.67 (66.7%)	0.70
Overripe (งอม/สุกจัด)	6	3	0.50 (50.0%)	0.50 (50.0%)	0.50
ภาพรวมเฉลี่ย (Macro / Overall)	40 ภาพ	32 ภาพ	0.71 (71.3%)	0.71 (70.7%)	Overall Acc: 80.00% (32/40)

6.2 การวิเคราะห์ข้อผิดพลาดและผลกระทบของ Class Imbalance

- ความโดดเด่นในคลาส Unripe:** โมเดลจำแนกมั่งคุดดิบได้ถูกต้องถึง **95.5% (21/22 ภาพ)** เนื่องจากสีเขียวของมั่งคุดดิบมีความแตกต่างทางสี (Chrominance) จากมั่งคุดสุกอย่างเด่นชัด
- รอยต่อระหว่าง Ripe และ Overripe:** ความคลาดเคลื่อนส่วนใหญ่เกิดขึ้นระหว่าง Ripe (8/12 ถูกต้อง) และ Overripe (3/6 ถูกต้อง) เนื่องจากในธรรมชาติ มั่งคุดระยะที่ 5 (ม่วงแดง) กำลังเปลี่ยนผ่านไปสู่ระยะที่ 6 (ม่วงดำเข้ม) ซึ่งเฉดสีเปลือกมีความใกล้เคียงกันมาก
- ผลกระทบของความไม่สมดุลของข้อมูล:** คลาส Overripe ในชุดข้อมูลดั้งเดิมมีจำนวนน้อยที่สุด ส่งผลให้โมเดลมีตัวอย่างภาพจริงของ Overripe ใน Test Set เพียง 6 ภาพ จึงสะท้อนค่า Recall ออกมาที่ 50%

7 การพอร์ตและนำไปใช้งานบน ESP32-S3 (Hardware Deployment)

ขั้นตอนสุดท้ายของการพัฒนาคือการแปลงโมเดล mangosteen_separable_cnn_94k_96x96_int8.tflite ให้กลายเป็นซอร์สโค้ดภาษา C และนำไปรันบนเฟิร์มแวร์ของบอร์ด LilyGO T-SIMCAM (ESP32-S3)

7.1 การแปลงโมเดลเป็น C Byte Array

แปลงไฟล์ไบนารี .tflite เป็น Header File mangosteen_model_data.h และกำหนดให้อยู่ในหน่วยความจำ Flash:

```
alignas(16) const unsigned char g_mangosteen_model_data[] = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, ...
};
const int g_mangosteen_model_data_len = 137328; // 134.11 KB ใน Flash
```

7.2 สถาปัตยกรรมระบบ TFLite Micro บน ESP32-S3

- การจัดสรร Tensor Arena ใน PSRAM:**
เนื่องจากโมเดลต้องการพื้นที่คำนวณชั่วคราว (Tensor Arena) ประมาณ **114.3 KB** ซึ่งการจองใน Internal SRAM อาจทำให้หน่วยความจำระบบของ ESP32-S3 ตึงตัว จึงใช้ฟังก์ชัน ps_malloc() จัดสรรพื้นที่ใน External Octal PSRAM ขนาด 8 MB ทำให้ระบบมีเสถียรภาพสูง ไม่เสี่ยงต่อปัญหา Stack/Heap Overflow
- MicroMutableOpResolver (Minimal Operator Footprint):**
ลงทะเบียนเฉพาะ Operators ที่โมเดลต้องใช้จริง 7 ตัว ได้แก่ AddDepthwiseConv2D, AddConv2D, AddAveragePool2D, AddReshape, AddFullyConnected, AddSoftmax, และ AddDequantize ช่วยลดขนาดคอมไพล์ของเฟิร์มแวร์ลงกว่า 300 KB
- ความเร็วในการประมวลผล (Inference Latency):**
การรันโมเดลบนแกนประมวลผล Xtensa LX7 ความเร็ว 240 MHz ใช้เวลาดำเนินการเฉลี่ย **5,667.2 มิลลิวินาที (~5.6 วินาทีต่อเฟรม)** ซึ่งมีความเหมาะสมอย่างยิ่งสำหรับการตรวจสอบผลมั่งคุดที่ผลในระบบคัดแยกแบบกึ่งอัตโนมัติหรือ Handheld Scanner

7.3 ระบบเว็บเซิร์ฟเวอร์และการแสดงผล (SoftAP Modern Web Interface)

เฟิร์มแวร์เปิดสัญญาณ Wi-Fi SoftAP ในชื่อ Mangosteen-AI เพื่อให้อุปกรณ์มือถือหรือแท็บเล็ตเชื่อมต่อเพื่อดูผลการทำนายได้ทันทีผ่าน Web Browser ที่ <http://192.168.4.1> โดยหน้าเว็บได้รับการออกแบบให้เป็น Responsive Two-Column Layout:

- คอลัมน์ซ้าย (Camera Feed):** แสดงภาพสดจากกล้อง OV2640 ความละเอียด 96 x 96 พิกเซล พร้อมกรอบเล็งเป้าผลมั่งคุด
- คอลัมน์ขวา (Inference Dashboard):** แสดงผลคลาสที่ทำนายได้ด้วยการเน้นสี (Unripe=เขียว, Ripe=แดงม่วง, Overripe=ม่วงเข้ม) พร้อมแถบเปอร์เซ็นต์ความน่าจะเป็น (Probability Bars) ทั้ง 3 ระดับ และการ์ดแสดงค่า Hardware Telemetry (FPS, Inference Latency ~5.6s, Free PSRAM, IP Address)

8 บทสรุปและทิศทางการพัฒนาต่อยอด (Conclusion & Next Steps)

การพัฒนาโมเดล Mangosteen_SeparableCNN_94k ได้บรรลุเป้าหมายสำคัญทุกประการ ทั้งในแง่ของความกะทัดรัด (94k พารามิเตอร์), การบีบอัดระดับสูง (Full INT8 ขนาด 134 KB), การคงความแม่นยำ (Val Acc 94.44%, Test Acc 80.00% โดยไม่สูญเสียความแม่นยำจากการ Quantize), และการนำไปรันบนฮาร์ดแวร์ ESP32-S3 ได้จริง

ข้อเสนอแนะในการยกระดับประสิทธิภาพในอนาคต:

- การเปิดใช้ ESP-NN Hardware Acceleration:** ในอนาคตสามารถลิงก์ไลบรารี esp-nn ของ Espressif เพื่อดึงชุดคำสั่ง SIMD / Vector Assembly ของ ESP32-S3 มาช่วยคำนวณ Depthwise Convolutions ซึ่งคาดว่าจะช่วยลดเวลา Inference ลงจาก 5.6 วินาที เหลือต่ำกว่า **1.5 - 2.0 วินาที**
- การขยายชุดข้อมูลช่วงรอยต่อ (Transition Stage Enrichment):** ถ่ายภาพมั่งคุดในระยะรอยต่อสีระหว่าง Ripe (ระยะ 4-5) และ Overripe (ระยะ 6) เพิ่มเติมภายใต้สภาพแสงที่หลากหลาย จะช่วยเพิ่มความแม่นยำของ Test Set จาก 80% ให้ขึ้นไปสู่ระดับ 90%+ ได้อย่างมีนัยสำคัญ